

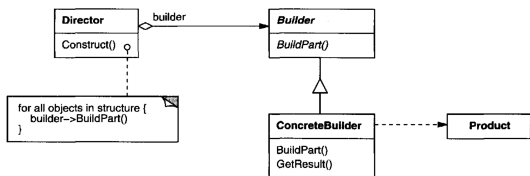
BUILDER PATTERN

The BUILDER pattern is a creational design pattern that lets you create complex objects step by step.

INTUITION BEHIND THE PATTERN!!

⇒ Suppose interviewer asks you to write a constructor with 10+ parameters; that means everytime a new object is created we have to pass a lot of params to avoid this we use BUILDER pattern.

STRUCTURE



- **Product** = The complex object being built.
- **Concrete Builder** = Implements the BUILDER interface; defines the specific representation of the product it builds.
- **Builder** = Interface that specifies steps (methods) for creating the Product object.
- **Director** = Construct the product object using the Builder interface following a specific steps.

IMP!!

The builder & factory pattern has a lot of similarities but differ in their use-cases which are imp to understand.

COMPARISON: ABSTRACT FACTORY VS BUILDER PATTERN

Feature	Abstract Factory Pattern	Builder Pattern
1. Intent	Create families of related/dependent objects.	Construct complex objects step by step.
2. Focus	Families of products (Product Suites).	Assembly of a single complex object.
3. Complexity	High (manages families).	Higher (manages construction steps).
4. Object Retrieval	Objects returned immediately.	Result returned at the very end.
5. Control	Client controls product family.	Director manages the process.
6. Analogy	Furniture factory (Victorian Chair, Victorian Sofa).	Custom meal assembly (Burger, Fries, Drink).

INTENT BEHIND THE PATTERN

⇒ Helps in separating construction of complex objects from its representation so that it can be re-used for other representation.

C++ CODE EXAMPLE

```
// 1. The Product class Computer {
public:
    std::string cpu;
    std::string ram;
    std::string storage;
    std::string gpu;

    void displaySpecs() const {}
};
```

→ The DIRECTOR can work with the BUILDER interface to create multiple flavours of products using the steps or methods defined.

```
// 2. The Abstract Builder Interface
class ComputerBuilder {
public:
    virtual ~ComputerBuilder() = default;
    virtual void buildCPU() = 0;
    virtual void buildRAM() = 0;
    virtual void buildStorage() = 0;
    virtual void buildGPU() = 0;
    virtual std::unique_ptr<Computer> getResult() = 0;
};
```

Based on the BUILDER class a lot of diff. types of concrete classes can be built like Gaming PC.

THE ANTI PATTERN

```
class Computer {
private:
    .....
public:
    // Giant constructor forcing you to pass everything, even if you don't care
    Computer(std::string c, std::string r, std::string s, std::string g, std::string ps, bool lc)
        : cpu(c), ram(r), storage(s), gpu(g), powerSupply(ps), liquidCooling(lc) {}

    void displaySpecs() const {
        std::cout << "CPU: " << cpu << ", RAM: " << ram << ", GPU: " << gpu << "\n";
    }
};
```

→ This is just one approach a lot of times we see the telescopic constructors as well means multiple parameterized constructors.

↑ This type of constructors are called FAT or GIANT constructors.

```
int main() {
    // Problem 1: What do all these strings mean without looking at the definition?
    // Problem 2: What if we don't want liquid cooling? We still have to pass 'false'.
    Computer myPc("Intel i9", "32GB", "2TB SSD", "RTX 4080", "850W", false);
}
```

* KEY ISSUES WITHOUT BUILDER PATTERN

1. The blind argument problem because of FAT constructors
2. Rigid initialization, as sometimes objects requires specific phases, validation checks or dependent params. (Happens a lot in real world.)
3. Difficult to maintain / defines open-close principles.

PROS of This pattern to mention:

1. Lets you vary a product's implementation
2. Isolates construction logic from representation.
3. Gives more control over construction logic.